
Where Does Manipulation Precision Come From?

Attributing the Precision Floor of Reinforcement-Learned Policies to the Action Interface

[TODO: Author name]

Independent Researcher, Tashkent, Uzbekistan

[TODO: email]

Draft status (8 Sep 2026). Every number printed in black is regenerated from a per-episode CSV committed in the public repository by `python reproduce.py`; none is transcribed from a console log. The eval-only part of Phase 0 (Sec. 6.1) is *complete* — 51 archived cells, 63 000 evaluation episodes, no training — and it is what the released checkpoints alone can say. Phase 1, Phase 2, the second environment and the retrained solver extremes require 73 training runs that have not been run; everything describing them is marked *blue*. [TODO: red] marks a decision that is still open. The August 2026 figures that appeared in earlier versions of this draft are superseded; `CHANGELOG.md` in the repository lists every correction.

Abstract

Reinforcement learning can now be pushed to millimetre precision on real robots: Luo et al. (2025) reach near-perfect success on connector insertion, timing-belt assembly and dual-arm coordination within one to two and a half hours of on-robot training, by putting a human in the loop and choosing the controller interface carefully. What is not known is which of those ingredients buys the precision. Xu et al. (2026) give the question a name, formalising the saturation point as a limit precision c in a data-scaling law for behaviour cloning and describing c as “an emergent property of the entire agent system,” but leaving open both what determines it and whether it survives online reinforcement learning. We study c for a reinforcement-learned pick-and-place policy on a 6-DoF arm in MuJoCo and test a single hypothesis: *the floor is set by the action interface* — the maximum per-step displacement, the joint-target smoothing constant, the control rate, and the joint-velocity limit imposed by the safety layer — rather than by the reward, the training budget, the observation encoder, or the contact-solver parameters of the simulator. We retrain policies at each interface setting instead of relabelling one policy’s errors, evaluate every cell with $n=2000$ episodes under fixed domain randomisation, run four control arms designed to falsify the interface hypothesis, and repeat the step-size axis in a second, independently authored environment — `gym-hil`, the open simulation companion to the human-in-the-loop line of work, with a different arm, a different controller and different interface constants — so that a result about interfaces is not a result about one robot. This version reports the complete eval-only phase. Two released policies with different training histories place the object with median terminal errors of 17.9 and 15.7 mm, roughly 0.5–0.6 times the 30 mm maximum step; the terminal error is almost entirely the tool-centre-point error at the instant the object leaves the fingers (r^2 up to 0.28) and essentially unrelated to how far the object then rolls ($r^2 \approx 0.004$), so the floor is set at release rather than by post-release settling. The contact-solver kill gate — the released policy re-evaluated under integration timestep, contact time constant, solver iteration count and impedance ratio changes spanning the range used by published MuJoCo manipulation work — moves c_{50} by at most 1.9 mm (a 2.7 mm total span across nine cells, against a nominal 16.4 mm), while changing how often the policy fails outright a good deal more. Evaluating all 38 stored checkpoints shows the floor is undefined until the task is solved, drops once, and then oscillates without improving over the last third of training; Xu et al.’s scaling form does not fit an RL checkpoint curve ($r^2 = 0.17$) and we report the failure rather than a fitted constant. A nested design of 200 scenes replayed under 10 noise streams each puts the intra-class correlation of the outcome at the floor at 0.175: five parts noise to one part scene. *The interface sweep, the estimated $c(\theta)$ and the training-based control arms are pending; see Sec. 6.* We release the environment, the trained checkpoints, per-episode evaluation logs with the sampled randomisation parameters, and a one-command reproduction script.

1 Introduction

Luo et al. (2025) showed that reinforcement learning can be driven to millimetre precision on real hardware — connector insertion, timing-belt assembly, dual-arm handovers — in one to two and a half hours of on-robot training. Their system changes several things at once: a human intervenes during learning, a reward classifier is trained from teleoperated samples, and the policy commands the arm through a carefully chosen impedance interface. Which of those is doing the work for *precision* specifically is not separable from that result, and it is not obvious. This paper isolates one candidate and measures it.

Start with the phenomenon. A learned manipulation policy that succeeds on 94.9% of pick-and-place episodes when “success” means landing within 45 mm of the target succeeds on 20.8% of them when the tolerance is 10 mm and on 5.3% when it is 5 mm ($n=2000$; Table 2). Nothing about the policy changed; only the ruler did. Every practitioner who has tightened a tolerance has seen this cliff, and yet the community’s benchmarks report a single success rate at a single, usually generous, tolerance (Jiang et al., 2026), which hides the cliff entirely.

Xu et al. (2026) were the first to characterise the cliff quantitatively. Training behaviour-cloning policies on three simulated tasks, they show that the number of demonstrations N needed to reach a precision P diverges as $\log N \propto 1/(P - c)$: there is a limit precision c that no amount of data crosses. They measure $c \approx 2.35$ mm for peg insertion and 2.75 mm for cuboid stacking, show that c moves when the wrist camera is removed ($\rightarrow 3.85$ mm), when the demonstrator is cleaner ($\rightarrow 1.27$ mm) or when randomisation is reduced ($\rightarrow 1.00$ mm), and conclude that c “is not a static physical constant of the task but an emergent property of the entire agent system.” They fix the action space to a 7-DoF end-effector delta pose throughout, study behaviour cloning only, and name reinforcement learning as the open next step. They release neither code nor data, so the RL case cannot be answered by re-analysing theirs; it has to be measured.

This paper asks the question their result leaves open, for reinforcement learning: **what component of the agent system sets c ?** We advance one falsifiable answer. In a delta-action interface the policy never commands a position; it commands a bounded displacement $a \in [-1, 1]^d$ that is scaled by a maximum step $\Delta_{\max}$, filtered by a smoothing constant α , rate-limited by a joint-velocity bound $\dot{q}_{\max}$ and applied at f Hz. These four numbers — the *action interface* $\theta = (\Delta_{\max}, \alpha, f, \dot{q}_{\max})$ — define the finest correction the arm can make in the final steps before release. Our hypothesis is that c is a function of θ and, to first order, of nothing else: not of the reward, not of the training budget, not of the observation encoder and not of the simulator’s contact parameters.

The hypothesis is worth stating because the obvious alternatives are all live. Gyenes et al. (2026) attribute the precision floor of imitation policies to the spectral bias of MLP encoders and remove it with Fourier features — a representation explanation. Bronars et al. (2026) show that reinforcement learning is agnostic to controller gains once hyperparameters are re-tuned — suggesting the learner can absorb interface changes, which would predict a flat $c(\theta)$. A precision term in the reward, or a longer training run, is what most practitioners try first. And any sim-only claim invites the objection that the floor is an artefact of contact stiffness in the MJCF. Each of these is a control arm in our design (Sec. 5), and the last of them — the one that would end the study — is reported here in full.

Three features of our setup make the question answerable on one workstation. First, the policy is trained by soft actor-critic (Haarnoja et al., 2018) on a state observation, so a full retraining costs hours rather than days and we can afford to *retrain* at every interface cell rather than relabel one policy’s errors post hoc — the latter cannot support any claim about training budget. Second, the environment records the sampled domain-randomisation vector of every episode, so each success curve is accompanied by a failure decomposition over the randomisation space (Sec. 5). Third, the step-size axis is repeated in a second environment we did not write: `gym-hil` (Aractingi et al., 2025), the open MuJoCo suite that accompanies the human-in-the-loop line of work, whose Franka Panda runs under operational-space control with a mocap target rather than a Piper under differential IK. Its interface constants were chosen independently of ours, and Sec. 6.2 reports what they are.

That comparison is worth stating before any result. The Piper environment of this paper commands at most 30 mm per step at 20 Hz with a smoothing constant of 0.45, and calls a placement successful within 45 mm. `gym-hil` commands at most 25 mm per step at 10 Hz with no smoothing at all, and calls a grasp successful within 50 mm and an arrangement within 30 mm. Two environments, two authors, two arms, two controllers — and every success tolerance in both lands between one and two commanded step sizes. If the precision floor were set by the reward, the budget or the simulator, there would be no reason for the people who wrote these environments to have converged on that ratio.

Contributions.

1. **(Measured.)** A complete eval-only characterisation of the floor of two released policies at $n=2000$: success-versus-tolerance curves with Wilson intervals over $\tau \in [5, 45]$ mm, c_{50} and c_{90} with bootstrap intervals, a decomposition of the terminal error into tool-centre-point error at release and post-release scatter, and a precision-versus-training-steps curve over all 38 stored checkpoints of both runs, and an intra-class correlation of the outcome from a nested scene-by-noise design.
2. **(Measured.)** A pre-registered contact-solver kill gate: the released policy re-evaluated under integration timestep, contact time constant, solver iteration and impedance-ratio changes, testing whether the floor is a simulator artefact before any training budget is spent on the main sweep.
3. **(Pending, 43 training runs.)** A retraining-based measurement of c across a one-factor-at-a-time sweep of the action interface, with the three remaining control arms — reward shaping, training budget and Fourier-feature encoding — and a hyperparameter-optimisation arm at the smallest step size.
4. An open environment, checkpoints, per-episode logs, an interface-flag test suite, and a reproduction script that regenerates every table and figure in this paper from the committed CSVs without a simulator run.

2 Related work

Precision limits of learned policies. Xu et al. (2026) introduce the limit precision c and the scaling law $\log N \propto 1/(P - c)$ for behaviour cloning with diffusion policies; the action interface is fixed and RL is left to future work. Gyenes et al. (2026) show that MLP-based imitation policies fail to learn high-frequency components of the state-to-action map and that Fourier feature encodings (Tancik et al., 2020) restore precision; we adopt their encoding as a control arm so that a representation explanation of our floor can be tested rather than assumed away. High-precision RL results exist. Luo et al. (2025) are the strongest: their human-in-the-loop system trains directly on a Franka and reaches near-perfect success on tasks with sub-millimetre clearances, with an average $2\times$ improvement over imitation baselines, in one to two and a half hours per task. Brown et al. (2026) reach sub-millimetre-clearance insertion with a hybrid position/force interface at 15 Hz. In both cases the interface is a design contribution rather than a swept variable, and in the first it is entangled with human intervention and a learned reward classifier. Our question is complementary and deliberately narrower: holding the human, the reward and the data collection fixed — indeed absent — how much of the terminal precision does the interface alone account for? A demonstration that a combination works cannot answer that; only a sweep can.

Action-space design. The most closely related work is Aljalbout et al. (2024), who train over 250 RL agents across 13 action spaces (joint/Cartesian $\times$ position/velocity/torque $\times$ one- and multi-step delta integrators) on reaching and pushing, at a fixed 60 Hz, and find that “an action space which yields or naturally limits the tracking error transfers better.” They vary the *type* of the interface; we hold the type fixed (Cartesian delta position through differential IK) and vary its *magnitude, filtering and rate*. Feng et al. (2026) sweep delta/absolute, joint/task, chunk length and execution horizon for imitation learning and recommend delta actions; Azimi et al. (2026) compare pose- and joint-space increments and velocities for vision-based RL and find joint velocity transfers best. Neither reports a precision floor or varies step size, smoothing or frequency. Bronars et al. (2026) re-tune hyperparameters for each PD-gain regime and find RL indifferent to gains “as long as the remaining hyperparameters are tuned to be compatible”; their frequency ablation (10–100 Hz) targets jitter in sim-to-real reaching. Their result is the strongest prior reason to expect $c(\theta)$ to be flat, and our hyperparameter-optimisation arm (Sec. 5) is designed to test exactly that. Action chunking in RL (Li et al., 2025; Chen et al., 2026) changes the effective control rate and temporal smoothness of the interface for long-horizon credit assignment; our frequency and smoothing axes isolate the same two quantities for precision.

Simulator fidelity as a confound. Aljalbout et al. (2026) warn that policies may succeed “simply by abusing modeling inaccuracies and simulator-specific corner cases”; Kaup et al. (2024) find that agents trained in some engines do not transfer to others and call the same-task-across-engines study missing. Hu et al. (2025) show a single unmodelled parameter (static friction) can dominate transfer. Domain randomisation is the standard mitigation (Muratore et al., 2022), though which parameters matter is usually settled by prior knowledge (Kaidanov et al., 2024). We do not attempt cross-engine transfer; we sweep the solver parameters most likely to shape terminal contact (timestep, constraint `solref`, impedance ratio, iteration count) and ask whether c moves. Sec. 6.1.3 reports the answer.

Evaluation statistics. Jiang et al. (2026) find that only 19.8% and 19.7% of state-of-the-art claims on LIBERO and SimplerEnv are provably significant, because aggregate success rates at small n cannot separate methods; Kress-Gazit et al. (2024) and Snyder et al. (2025) argue for interval estimates and sequential tests, and Patterson et al. (2024) show that five seeds are almost never enough for strong claims. This is not an abstract concern for us: an earlier version of this draft reported, from $n=200$, that our two released policies were indistinguishable below a 15 mm tolerance (15.0% vs. 15.5% at 10 mm, $p=0.89$) and used that as the motivating observation. At $n=2000$ the same comparison gives 17.5% vs. 20.8% ($z=2.65$, $p=0.008$): the null was an artefact of the sample size, and the motivating sentence has been rewritten (Sec. 6.1.1). We therefore evaluate $n=2000$ episodes per cell with Wilson intervals (Wilson, 1927) and three training seeds, and report interquartile means over seeds (Agarwal et al., 2021).

Human-in-the-loop RL and its open simulation companion. The practical route to Luo et al. (2025)’s result for most groups is `gym-hil` (Aractingi et al., 2025) and the LeRobot port of the method, which provide MuJoCo Franka tasks with the same capped delta-action interface and the same human-intervention hooks. We use `gym-hil` not as a baseline to beat but as a second measuring instrument: it is the closest public analogue of our setup written by other people, and repeating one axis of the sweep in it is the cheapest available guard against a result that is really about the Piper. Sec. 6.2 reports its released interface constants, which are not stated in its documentation and which we read out of the code.

What could not be reused. We searched for public data that would answer any part of this question without new rollouts and found none. Manipulation evaluation harnesses discard the continuous quantity at the moment they threshold it: ManiSkill3’s evaluation returns booleans (the goal distance is computed against a fixed threshold and dropped), `robomimic` returns aggregate success rates with no per-episode dump, and LIBERO exposes only a sparse reward. Demonstration datasets store object poses and therefore contain terminal errors, but for demonstrations, not for policy rollouts. Xu et al. (2026) release nothing; Feng et al. (2026) promise code on publication and have not yet released it. Consequently every success-versus-tolerance curve in this paper had to be generated by re-running policies, and we release ours per-episode so that the next study does not.

3 Problem statement

Task and interface. An episode of the pick-and-place task samples an object pose and a target position, both drawn from randomised annuli in front of the arm (Sec. 4). The policy π_ϕ observes a 51-dimensional state and outputs $a_t \in [-1, 1]^5$: a Cartesian displacement of the tool centre point, a wrist-yaw increment and a gripper command. The environment maps a_t to a joint target through the interface

$$\Delta x_t = \Delta_{\max} a_t^{xyz}, \quad q_t^{\text{raw}} = \text{IK}(x_{t-1} + \Delta x_t), \quad q_t^{\text{cmd}} = (1-\alpha) q_{t-1}^{\text{cmd}} + \alpha q_t^{\text{raw}}, \quad \|q_t^{\text{cmd}} - q_{t-1}^{\text{cmd}}\|_\infty \leq \dot{q}_{\max}/f, \quad (1)$$

executed by position actuators at f Hz. We call $\theta = (\Delta_{\max}, \alpha, f, \dot{q}_{\max})$ the action interface. In the released environment $\theta_0 = (30 \text{ mm}, 0.45, 20 \text{ Hz}, 1.0 \text{ rad s}^{-1})$. Two properties of Eq. 1 are easy to misread and matter for the design. First, the smoothing acts on *joint targets after IK* — it is a first-order low-pass filter on the commanded trajectory, not on the action. Second, the joint-velocity clamp is part of a safety layer shared with the hardware interface, and it is not slack: over the $n=2000$ baseline evaluations it is active on 78% (π_{v2}) and 83% (π_{v1}) of control steps. It is therefore a binding constraint at θ_0 and must be swept jointly with $\Delta_{\max}$ rather than held fixed (Sec. 5).

Precision and the floor. The terminal placement error is $e = \|p_{xy}^{\text{obj}} - p_{xy}^{\text{tgt}}\|_2$ once the object has settled after release. For a tolerance τ the success rate is $S(\tau) = \Pr[e \leq \tau \wedge \text{lifted} \wedge \text{settled}]$; an episode that never lifted or never settled counts as a failure at every tolerance, so that a dropped object lying 0.1 mm from the target is not scored as a 0.1 mm success. Because e is logged per episode, a single evaluation yields $S(\tau)$ for every τ at once, and `tests/test_tolerance_semantics.py` asserts that reading the curve post hoc agrees exactly with thresholding at that τ . We use two estimators of the floor. The *plateau estimator* c_{50} is the tolerance at which $S(\tau)$ crosses 50%; it is model-free, and equals the median terminal error whenever the valid fraction exceeds one half. The *scaling-law estimator* $\hat{c}$ fits Xu et al.’s form to precision-versus-training-steps curves obtained by evaluating every stored checkpoint of a training run, replacing demonstrations N by environment steps; it is comparable to their c but model-dependent. We also report c_{90} , the 90th-percentile tolerance, which is more sensitive to the tail and correspondingly noisier.

Hypotheses.

H1 (interface). c is a monotone function of the effective per-step resolution of θ . Under a one-factor-at-a-time sweep, c_{50} moves with $\Delta_{\max}$, α and f by more than its 95% interval.

H0a (reward). Adding a precision term to the terminal reward moves c .

H0b (budget). Training three times longer at fixed θ_0 moves c .

H0c (representation). Fourier-feature encoding of the observation moves c at fixed θ_0 .

H0d (simulator). Changing timestep, `solref`, impedance ratio or solver iterations within ranges used by published MuJoCo manipulation work moves c at fixed θ_0 .

H0e (release physics). The floor is dominated by object scatter between release and settling rather than by the TCP error at the moment of release.

H1 is supported if it holds and H0a–H0e fail. If H0d holds, the paper’s claim collapses to “the floor is a simulator artefact” and we say so; this is the pre-registered kill gate. If H0c holds, the representation explanation of Gyenes et al. (2026) extends to RL and H1 must be restated as a joint interface–representation account. H0d and H0e are settled in Sec. 6.1; H0a–H0c and H1 require the training campaign.

4 Experimental setup

Environment. The arm is an AgileX Piper (6 revolute joints plus a parallel gripper) modelled in MuJoCo (Todorov et al., 2012) from the Menagerie-derived MJCF, with `implicitfast` integration, elliptic friction cones, `impratio`=10 and otherwise default solver settings (timestep 2 ms, Newton solver, 100 iterations, 25 physics sub-steps per control step at 20 Hz). Gravity compensation is enabled on all links. Position actuators use k_p =80, k_v =5 on joints 1–3, k_p =40 on joint 4, k_p =10, k_v =1.5 on joints 5–6 and k_p =200 on the gripper. The object is a cylinder of radius 24 mm and height 70 mm, mass 60 g, with contact parameters `solref`=(0.010, 1.2), `solimp`=(0.92, 0.97, 0.001) and friction (1.5, 0.05, 0.005); finger pads use `solref`=(0.012, 1.4). The target is a site on the table; success requires the object within τ_{xy} of the target and 20 mm in height, at rest (speed < 0.05 m/s), for three consecutive control steps with the gripper open. The default tolerance is τ_{xy} =45 mm; all curves below are evaluated post hoc at every τ .

Observation and action. The 51-dimensional state contains normalised joint positions and velocities, gripper state, TCP position and approach vector, object and target positions, object velocity, relative vectors, grasp flags, the previous action and the sampled object size. The 5-dimensional action is described in Eq. 1; differential IK uses damped least squares (damping 0.06, step 0.6, two iterations) with a position-first null-space projection.

Domain randomisation and noise. Per episode: object radial distance 0.30–0.39 m and azimuth $[-0.73, -0.17]$ rad, target on the mirrored annulus; object radius and height scale $[0.85, 1.15]$, mass 30–120 g, friction 0.8–2.0; table friction 0.6–1.4; actuator k_p and k_v scales $[0.8, 1.25]$ per actuator; joint friction-loss scale $[0.5, 1.6]$; initial joint jitter 0.06 rad; lighting. Observation noise is additive Gaussian (joint position 3 mrad, object position 6 mm, target 4 mm, object rotation 0.05 rad) with one step (50 ms) of observation latency, 4 mrad action noise and 1% command dropout. A “hard corner” fraction oversamples the far-radius region during fine-tuning. **Correction:** the environment also draws a $\pm 40\%$ joint-damping scale, and earlier descriptions of this work listed it among the randomised dynamics. The released MJCF sets no joint damping, so that scale multiplies zero and is a no-op. The randomiser now reports the drawn scales together with a `joint_damping_is_noop` flag, and the flag is true for the released model; the parameter is excluded from the randomisation count and from the failure attribution. Adding nominal damping to the MJCF would change the dynamics of every result in this paper and was therefore not done after the evaluations began.

Learning algorithm. Soft actor-critic (Haarnoja et al., 2018) from Stable-Baselines3 (Raffin et al., 2021): two 256-unit hidden layers, learning rate 3×10^{-4} (fine-tuning 10^{-4}), batch 256, replay buffer 4×10^5 , γ =0.98, τ =0.01, automatic entropy tuning initialised at 0.1, 12 parallel environments, 12 gradient steps per environment step, 5 000 warm-up steps. A reverse curriculum samples episode starts from later task phases with decreasing probability during training and is disabled at evaluation. The reward combines dense reaching, lifting and transport shaping with stage bonuses (touch 3, grasp 25, lift 25, over-target 12, success 200) and per-step penalties on time, action magnitude, action change, joint speed, collision and unsafe commands. A terminal precision term $w_{\text{prec}}(1 - e/\tau_{xy})$ exists and is *disabled* (w_{prec} =0) in both released checkpoints. Measured throughput on the development machine (CPU-only training) is

Table 1: Cells. Every training cell is run with three seeds and evaluated with $n=2000$ episodes. “Eval-only” cells re-evaluate an existing checkpoint under a changed simulator or tolerance and cost no training. Run-time estimates assume 6 h per Piper training run; the `gym-hil` cells are roughly an hour each. Phase 0 is complete; Phases 1–2 are pending.

Phase	Arm	Cells	Purpose	Runs	Status
0	Solver (kill gate), eval-only	timestep $\{1, 2, 4\}$ ms; <code>solref</code> scale $\{0.5, 1, 2\}$; iterations $\{20, 100\}$; <code>impratio</code> $\{1, 10\}$; two joint extremes	does the existing floor move?	0	done
0	Checkpoint curve, eval-only	all 38 stored checkpoints of both runs	$\hat{c}$ vs. training steps; H0b	0	done
0	Release decomposition, eval-only	π_{v1}, π_{v2}	H0e	0	done
0	Nested noise replay, eval-only	200 scenes $\times$ 10 noise streams, π_{v2}	ICC; failure attribution	0	done
0	Solver (kill gate), retrain	two extreme settings	H0d	6	pending
1	$\Delta_{\max}$	$\{5, 10, 20, 30\}$ mm, with $\dot{q}_{\max}$ scaled so the clamp binds equally	H1	12	pending
1	α	$\{0.2, 0.45, 1.0\}$ (1.0 = no smoothing)	H1	6	pending
1	f	$\{10, 50\}$ Hz	H1	6	pending
1	$\dot{q}_{\max}$ alone	$\{0.5, 2.0\}$ rad/s at $\Delta_{\max}=30$ mm	disentangle clamp from step	6	pending
1	2nd env. <code>gym-hil</code>	$\Delta_{\max} \in \{5, 10, 25, 50\}$ mm, $\alpha \in \{0.2, 0.45\}$, $f \in \{5, 25\}$ Hz	is H1 about interfaces or about the Piper?	24	pending
2	Reward	$w_{\text{prec}}=60$ from scratch	H0a	3	pending
2	Budget	$3\times$ steps at θ_0	H0b	1	pending
2	Fourier features	θ_0 with Fourier-encoded state	H0c	3	pending
2	HPO	10 Optuna trials at $\Delta_{\max}=5$ mm	Bronars et al. (2026) objection	10	pending
Total training runs				77	

26–62 environment steps per second; a 1.2 M-step run takes 5–6 h. **[TODO: Re-measure throughput with SAC updates on the GPU before the sweep; the sweep budget below assumes ≤ 6 h per run.]**

The two released policies, and what separates them. π_{v1} is the *best*-checkpoint selection of a 1.0 M-step run (runs/`sac_full`); π_{v2} is the *final* model of a 0.3 M-step fine-tune of it (runs/`corner_ft`) with 35% hard-corner oversampling and learning rate 10^{-4} . The reward is identical in both ($w_{\text{prec}}=0$); an earlier description of this work stated that they differed by reward, and that was wrong. Any π_{v1} -vs- π_{v2} difference therefore confounds three things — the shifted sampling distribution, the extra 0.3 M steps, and the checkpoint-selection rule — and none of them is the reward. This is exactly why the sweep in Sec. 5 retrains every cell, including the θ_0 cell, under one protocol with final-checkpoint selection, rather than reusing either released policy as a baseline.

Evaluation protocol. Deterministic policy, randomisation and noise on, curriculum off, episode i seeded with $s_0 + i$. Every episode writes one CSV row: seed, git hash, success, grasp, lift, termination reason, terminal error e , TCP–target error at the release step, post-release scatter, episode length, collision steps, safety clamp count and rate, the resolved interface and solver settings, and the full sampled randomisation vector. The release step is detected physically, as the last control step on which both finger pads are in contact with a lifted object, rather than from the gripper command — a command-based detector fires on transient open commands during transport and inflates both terms of the decomposition. Success curves $S(\tau)$ are reported with 95% Wilson intervals (Wilson, 1927); differences between cells are tested with a two-proportion z -test at the tolerance of interest and reported with effect size in percentage points and a 95% interval; c_{50} and c_{90} carry a 2000-draw percentile bootstrap over episodes. Each training cell will be run with three seeds; we report the per-seed curves and the interquartile mean.

Instrumentation as a first-class artefact. Two test files guard the sweep. `tests/test_interface_flags.py` asserts, for every swept quantity, that the flag reaches the simulator: that $\Delta_{\max}$ bounds the one-step displacement and is monotone in reachable displacement, that the control rate changes the number of physics sub-steps (10/20/50 Hz $\rightarrow$ 50/25/10) and the safety clamp $\dot{q}_{\max}/f$, that $\dot{q}_{\max}$ sets that clamp, that the solver overrides land on `model.opt` and on the object and finger-pad `solref` but not on the table, and that scaling `solref` preserves the damping ratio. Without these, a mis-wired flag would make several cells of Figure 6.3 the same experiment run repeatedly, and nothing in the printed output would say so. One test documents a real defect rather than asserting correctness: at 20 Hz a 4 ms timestep is 12.5 sub-steps and silently rounds to 12, a 4% shortfall in simulated time per control step. The eval-only gate reports the 4 ms cell as measured; the retrained solver extreme uses 2.5 ms, which divides exactly.

5 Experimental design

The design is one-factor-at-a-time around θ_0 , with retraining in every cell. Table 1 lists the cells. Phases are ordered so that the experiment most likely to kill the hypothesis runs first, and so that everything obtainable without training is obtained first.

Table 2: Success rate versus tolerance for the two released policies, $n=2000$ evaluation episodes each (seeds 10000–11999), domain randomisation and observation noise on, deterministic actions, curriculum off. 95% Wilson intervals; c_{50} with a 2000-draw percentile bootstrap.

policy / cell τ (mm)	45	35	30	25	20	15	10	7.5	5	c_{50} (mm)
π_{v1}	93.2 [92.0, 94.2]	88.9 [87.4, 90.2]	82.4 [80.7, 84.0]	70.9 [68.9, 72.8]	54.3 [52.1, 56.5]	35.3 [33.2, 37.4]	17.5 [15.9, 19.3]	9.6 [8.3, 10.9]	4.5 [3.7, 5.5]	18.7 [18.1, 19.4]
π_{v2}	94.9 [93.8, 95.8]	93.3 [92.1, 94.3]	90.6 [89.3, 91.8]	82.8 [81.1, 84.4]	67.8 [65.7, 69.8]	43.8 [41.6, 46.0]	20.8 [19.1, 22.7]	12.2 [10.8, 13.7]	5.3 [4.4, 6.4]	16.1 [15.7, 16.6]

Phase 0: kill gate and free measurements. Before any training, the released policy π_{v2} is re-evaluated with each solver setting changed in the loaded model (timestep and iterations through `model.opt`, contact parameters by scaling `geom_solref` on the object and the four finger-pad geoms, preserving the damping ratio). If the released policy’s c_{50} moves by more than its interval under a solver change, we retrain at the two extreme settings; if the retrained floor moves as far as it does under a $30 \rightarrow 10$ mm step change, the study stops here and the write-up becomes a negative result about simulator dependence. The same phase evaluates every stored checkpoint of both runs to obtain precision as a function of training steps, and decomposes each terminal error into TCP-at-release error and post-release scatter. Sec. 6.1 reports all three.

Phase 1: interface sweep. Cells in $\Delta_{\max}$ are run with $\dot{q}_{\max}$ scaled proportionally ($30 \text{ mm} \rightarrow 1.0$, $20 \rightarrow 0.67$, $10 \rightarrow 0.33$, $5 \rightarrow 0.17$ rad/s), because at θ_0 the joint-velocity clamp binds on about four steps in five (Sec. 6.1.1) and a step-size sweep with a fixed clamp would measure the clamp. A separate two-cell sweep of $\dot{q}_{\max}$ at fixed $\Delta_{\max}$ separates the two, and the clamp rate is reported per cell. When f changes, the number of physics sub-steps, the observation latency in milliseconds and the episode budget in steps all change; we hold latency and budget fixed in seconds, so only the sub-step count and the clamp — both parts of the interface — move with f . At 50 Hz a 30 mm step is 1.5 m s^{-1} , above the safety layer’s Cartesian rate limit of 0.6 m s^{-1} ; that cell raises the limit to 1.6 m s^{-1} so that a second, unswept limiter does not silently bind, and the change is reported. Domain randomisation ranges are held fixed at their released values in all cells, since Xu et al. (2026) show c moves with randomisation range.

Phase 2: controls. The reward arm trains from scratch with the precision term enabled, rather than fine-tuning, so that budget and reward are not confounded. The budget arm continues the baseline to 3.6 M steps. The Fourier arm replaces the state input by $[s, \sin(2\pi Bs), \cos(2\pi Bs)]$ with a fixed Gaussian B following Gyenes et al. (2026), using the same B in every parallel environment and at evaluation. The HPO arm re-tunes learning rate, entropy coefficient and batch size at the smallest step size using Optuna (Akiba et al., 2019); if tuned hyperparameters recover the θ_0 floor at $\Delta_{\max}=5$ mm, the Bronars-style objection stands and we report it, and the claim weakens to “the interface sets the floor for a fixed optimiser configuration.”

Failure attribution. For every cell we fit a cross-validated logistic model and a gradient-boosted classifier of failure at $\tau=c_{50}$ on the sampled randomisation vector and initial state, and report AUC and permutation importances. A nested design (1000 fixed randomisation draws $\times$ 10 noise re-seeds) estimates the intra-class correlation of outcomes within a draw, separating environment-determined from irreducible failures. *If AUC < 0.6 and ICC < 0.1 in every cell, the failures are noise and this section reduces to a paragraph.*

6 Results

6.1 Phase 0, measured: what the released checkpoints already say

Everything in this subsection was obtained without training a single policy. It uses only the checkpoints committed in the repository, and it is reproduced end to end by `python reproduce.py` from the archived per-episode CSVs.

6.1.1 The floor of the two released policies

Table 2 gives the full success-versus-tolerance curve for both released policies at $n=2000$ episodes on a common seed block, with 95% Wilson intervals, and c_{50} with a 2000-draw bootstrap. Four things follow.

Table 3: H0e: decomposition of the terminal error into the tool-centre-point error at the moment the object leaves the fingers and the distance the object then travels before it settles. Medians over episodes in which the object was lifted and released.

policy	n	TCP err. at release	post-release scatter	terminal e	$\rho(\text{TCP}, e) / \rho(\text{scatter}, e)$
π_{v1}	1899	21.8	9.5	17.9	0.53 / -0.06
π_{v2}	1908	17.7	7.6	15.7	0.35 / 0.07

The cliff is steep and it is not near zero. π_{v2} succeeds on 94.9% of episodes at $\tau=45$ mm and on 5.3% at $\tau=5$ mm; its c_{50} is 16.1 [15.7, 16.6] mm and its c_{90} is 28.9 mm. The floor sits at roughly half the 30 mm maximum step and at 1.4 times the effective per-step reach of the joint-velocity clamp, both of which are consistent with H1 and neither of which is evidence for it: they are the quantities the sweep must separate.

The joint-velocity clamp is the binding interface term at θ_0 . It fires on 78% of the control steps of an average π_{v2} episode and 83% of a π_{v1} episode. A step-size sweep that held $\dot{q}_{\max}$ fixed would therefore be measuring the clamp for most of its range, which is why the $\Delta_{\max}$ cells scale $\dot{q}_{\max}$ with them and a separate axis varies $\dot{q}_{\max}$ alone.

The motivating null of the earlier draft does not survive $n=2000$. An earlier version of this work reported that the two policies were indistinguishable below 15 mm (15.0% vs. 15.5% at $\tau=10$ mm, $n=200$, $p=0.89$) and used that as the observation motivating the whole study. At $n=2000$ the difference at 10 mm is +3.3 percentage points [+0.9, +5.7], $z=2.65$, $p=0.008$, and it is still significant at 7.5 mm (+2.6 pp, $p=0.007$); it disappears only at 5 mm (+0.9 pp, $p=0.21$). The two c_{50} intervals, [18.1, 19.4] and [15.7, 16.6] mm, do not overlap. Training history *does* move this floor. What survives is the weaker and more interesting statement that the *effect* of training history decays monotonically as the tolerance tightens — +13.5 pp at 20 mm, +8.5 at 15, +3.3 at 10, +2.6 at 7.5, +0.9 at 5 — so the two histories converge as the ruler approaches the floor without ever having been equal. We report this because it is the clearest illustration in the paper of the point Jiang et al. (2026) and Snyder et al. (2025) make: at $n=200$ we would have published a null that is not there.

The comparison is confounded anyway. π_{v1} and π_{v2} differ by a shifted sampling distribution, 0.3 M extra steps and the checkpoint-selection rule (best vs. final). No causal reading of the difference is available, which is precisely why every cell of Phase 1 is retrained under a single protocol.

6.1.2 H0e: the floor is set at release, not by post-release settling

If the terminal error were dominated by how far the object rolls after the fingers open, no change to the action interface could move it and H1 would be irrelevant. Table 3 decomposes the error. For π_{v2} the median tool-centre-point error at the instant the object leaves the pads is 17.7 mm (p90 27.9), the median distance the object subsequently travels before settling is 7.6 mm (p90 17.0), and the median terminal error is 15.7 mm. The terminal error correlates with the TCP error at release ($r=0.35$ for π_{v2} , 0.53 for π_{v1}) and is uncorrelated with the post-release scatter ($r=0.07$ and -0.06 ; $r^2 \approx 0.004$). **H0e is rejected:** the object does move after release, but where it ends up is set by where the gripper let go, not by the rolling. Terminal precision in this task is a placement problem at the last held step, which is the step the action interface governs.

6.1.3 H0d: the contact-solver kill gate

Table 4 re-evaluates π_{v2} under nine contact-solver settings spanning the range used by published MuJoCo manipulation work: integration timestep 1, 2 and 4 ms; contact time constant of the object and finger pads scaled by 0.5, 1 and 2 with the damping ratio held; Newton iterations 20 and 100; impedance ratio 1 and 10; and the two joint extremes (1 ms with $\text{solref} \times 0.5$, 4 ms with $\text{solref} \times 2$). Each cell is $n=1000$ on a common seed block.

c_{50} moves by at most 1.9 mm, a total span of 2.7 mm across the nine cells against a nominal 16.4 [15.7, 17.0] mm — about 17% of the floor. The movement is not uniformly negligible: two cells, $\text{solref} \times 0.5$ (18.3 [17.5, 19.1] mm) and the stiff/fine extreme (17.1 [16.5, 18.0] mm), have intervals that exclude the nominal point estimate. Softening the contact time constant makes the floor worse and, more sharply, degrades the tail: $S(45)$ falls from 95.8% to 88.6% and c_{90} rises correspondingly, while c_{50} moves by under 2 mm. In other words the solver mostly changes how often the policy fails outright, not how precisely it places when it succeeds.

Table 4: Phase 0 kill gate (H0d), eval-only. The released policy π_{v2} re-evaluated under each contact-solver setting, $n=1000$ episodes per cell on a common evaluation seed block. Δc_{50} is relative to the nominal solver; a cell moves the floor only if its interval excludes the nominal point.

solver cell	$S(45)$ %	median e (mm)	c_{50} (mm) [95% CI]	Δc_{50}
nominal	95.8	16.0	16.4 [15.7, 17.0]	+0.0
dt1ms	95.2	15.6	15.9 [15.4, 16.5]	-0.5
dt4ms	91.3	16.1	16.9 [16.3, 17.5]	+0.4
solref0.5	88.6	17.0	18.3 [17.5, 19.1]	+1.9
solref2	95.5	15.8	16.2 [15.6, 16.8]	-0.2
iter20	95.8	16.0	16.4 [15.7, 17.0]	+0.0
impl	91.4	15.8	16.6 [16.0, 17.4]	+0.1
extremeA	90.0	16.1	17.1 [16.5, 18.0]	+0.6
extremeB	93.9	15.2	15.6 [15.1, 16.5]	-0.8

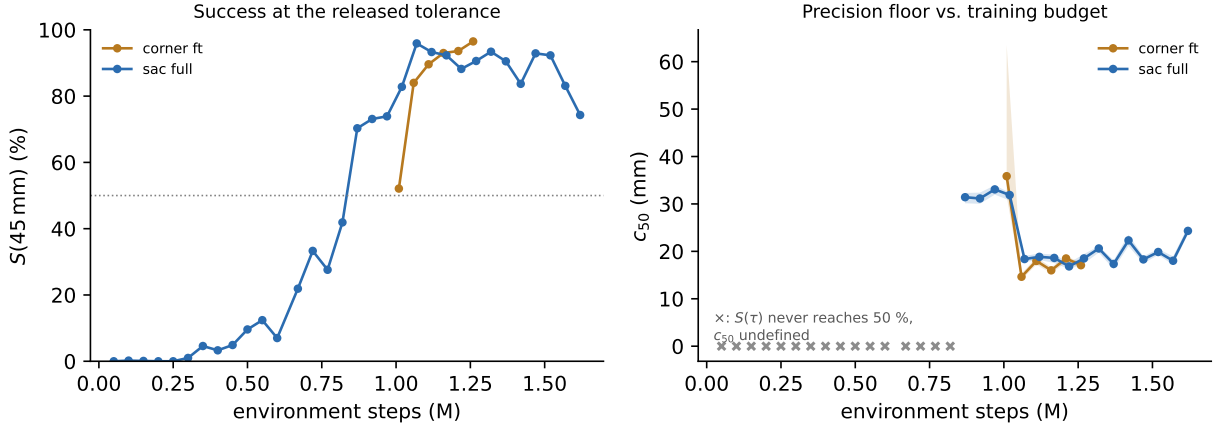


Figure 1: Every stored checkpoint of both runs, $n=1000$ episodes each (38 checkpoints, 38 000 episodes, no training). Left: success at the released 45 mm tolerance. Right: c_{50} , which is undefined ($\times$) until the policy succeeds on more than half of episodes; shaded bands are bootstrap 95% intervals. The floor is a property of the converged policy: it is undefined for the first 0.82 M steps, drops sharply once the task is solved, and then does not improve.

Two negative details are worth stating because they would otherwise look like bugs. The 20-iteration cell is *bit-identical* to the 100-iteration cell in all 1000 episodes; a direct probe (`results/precision/solver_iterations_probe.md`) shows the knob is live — 5 iterations changes the trajectory, 2 or 1 breaks the simulation entirely, with median errors above 450 mm — and that the Newton solver simply converges in fewer than 20 iterations for this contact configuration. Iteration count in $[20, 100]$ is an identity here, not a measured null. Separately, at 20 Hz a 4 ms timestep is 12.5 sub-steps and rounds to 12; that cell is reported as measured and the retrained extreme uses 2.5 ms instead.

Gate decision. The pre-registered rule was: if c_{50} moves by more than its interval under any eval-only solver cell, retrain at the two extreme settings before proceeding, and abandon the interface claim if the retrained floor moves as far as a $30 \rightarrow 10$ mm step change does. Two cells moved by more than their interval, so **the gate escalates to the retraining stage**: six runs (two extremes $\times$ three seeds) that have not been performed. The eval-only bound is nonetheless informative — a 1.9 mm solver effect is the number the interface effect must beat, and if the $\Delta_{\max}$ axis turns out to move c_{50} by, say, 2 mm, the paper has no result regardless of what the retrained extremes do. **[TODO: Run the six retraining runs and fill this decision. Do not run Phase 1 before it.]**

Table 5: The two environments’ released action interfaces, read out of the code rather than from documentation. Neither project states these numbers in its README, and in `gym-hil` the base class’s signature (50 Hz) disagrees with the task environment’s default (10 Hz). The last row is the observation that motivates H1: three tasks written by two groups for two arms all set their success tolerance between one and two commanded step sizes.

	this paper (Piper)	<code>gym-hil</code> (Franka)
controller	differential IK + safety layer	operational-space, mocap target
$\Delta_{\max}$ (commanded)	30 mm	25 mm
$\Delta_{\max}$ (realised in one step)	—	≈ 10 mm (0.40 $\times$)
f	20 Hz	10 Hz
physics sub-steps	25×2 ms	50×2 ms
α	0.45	1.0 (none)
$\dot{q}_{\max}$	1.0 rad/s, binds on 78–83% of steps	none
episode budget	200 steps = 10 s	100 steps = 10 s
success tolerance	45 mm (placement)	50 mm (grasp), 30 mm (arrangement)
tolerance / $\Delta_{\max}$	1.5	2.0, 1.2

6.1.4 H0b, without training anything: precision versus training budget

Figure 1 evaluates all 38 stored checkpoints — 32 of the baseline run spanning 50 k to 1.62 M environment steps and 6 of the fine-tune — at $n=1000$ each. It is the RL analogue of Xu et al.’s data-scaling curve with environment steps in place of demonstrations, and it costs no training at all.

c_{50} does not improve with budget once the task is solved. For the baseline run it is undefined below 0.82 M steps (success never reaches 50%), first measurable at 31.4 mm at 0.87 M, falls to 16.8 mm at 1.22 M, and then *oscillates between 16.8 and 24.3 mm* for the remaining 0.4 M steps, finishing at 24.3 [23.7, 25.2] mm at 1.62 M with success at 45 mm falling from 96.5% to 74.3%. Over the last third of the run the floor moves by more than its own bootstrap interval in both directions and ends *worse* than its best. This is a within-run, single-seed observation and it does not replace the pre-registered 3 $\times$ -budget arm, which is trained from scratch — but it is the first evidence on H0b and it points the same way: past the point where the task is solved, more environment steps buy noise, not precision.

Two consequences for how the released policies should be read. First, π_{v2} ($c_{50}=16.1$ mm) is not the best checkpoint of its own fine-tune: the checkpoint at 1.06 M steps reaches 14.6 [14.0, 15.3] mm. Second, the first fine-tune checkpoint, immediately after the baseline weights are loaded and the replay buffer is empty, sits at 35.9 mm — a large transient dip that recovers within 50 k steps. A campaign that selected checkpoints by a 15–50-episode evaluation callback, as the released runs did, is selecting from this much noise, which is why the sweep in Sec. 5 fixes the final checkpoint as the reported one.

Xu et al.’s functional form does not fit. Fitting $\log(\text{steps}) = a/(P - c) + b$ with $P = 1/c_{50}$ over the checkpoints that solve the task gives $\hat{c} = 18.4$ mm with $r^2 = 0.17$ for the baseline run and an uninterpretable fit ($r^2 \approx 0$, six points) for the fine-tune. We report the failure rather than the number. A behaviour-cloning data-scaling curve is monotone by construction — more demonstrations never hurt — whereas an RL checkpoint curve is a stochastic process that can and does go backwards, so the model-dependent estimator $\hat{c}$ is not usable in this setting and the model-free c_{50} is the estimator we report throughout. That is itself a finding about transporting their result to RL, and it is one we would have missed by reporting a fitted constant without its r^2 .

6.2 The second environment, and what its released interface is

The step-size axis is repeated in `gym-hil` (Aractingi et al., 2025), the MuJoCo suite that accompanies the human-in-the-loop line of work. Its Franka Panda is driven by operational-space control towards a mocap target, not by differential IK with a joint-velocity safety clamp; it was written by different people for a different robot; and it is Apache-2.0, so the comparison is reproducible by anyone.

Table 5 reports the constants. Two of them are worth calling out because they are not documented anywhere and a user of the package would not know them. First, the 25 mm cap lives in `EEActionWrapper` and applies only to the wrapped environment ids; the `Base` ids apply no scaling at all, so an action of 1.0 commands a one-metre mocap

displacement, clipped to the workspace box. Second, the *realised* displacement of a saturated step is about 40% of the commanded cap — roughly 10 mm for the released 25 mm setting — because operational-space control does not reach the mocap target within one 100 ms control step. Any comparison of the two environments’ step sizes has to use the realised number; the ratio is linear in $\Delta_{\max}$ and is pinned by a test (`tests/test_gymhil_interface.py`) so that a future release of the package cannot change it silently.

Which task, and an honest reason. The natural analogue of our placement task is `PandaArrangeBoxes`, which has a real 30 mm placement tolerance. As released it is not learnable from state: success is a conjunction over five blocks, while the state observation is three numbers — the position of block 1. The other four blocks and all five targets are unobservable. We therefore sweep `PandaPickCube` and take the terminal gripper-to-block distance as the precision measure, which is exactly the quantity `gym-hil`’s own success condition thresholds at 50 mm. That is a reach-and-grasp precision rather than a placement precision, and the difference is a limitation of the comparison rather than a feature of it. A test records the defect so that the task choice is revisited if a future release fixes it.

Results: $S(\tau)$ and c_{50} for $\Delta_{\max} \in \{5, 10, 25, 50\}$ mm on `PandaPickCube`, three seeds, $n=2000$ each, with the α and f axes as secondary cells. If c_{50} tracks $\Delta_{\max}$ in both environments, H1 is a statement about delta-action interfaces; if it tracks it in one, H1 is a statement about the Piper and the paper says so. These cells are cheap — throughput is 63 environment steps per second on two CPU cores against 26–62 for the Piper with 12 gradient steps, and the budget is 300 k steps rather than 1.2 M, so a three-seed step axis is under a day of wall clock. **[TODO: Run and fill; 24 runs via `run_sweep.py -env gymhil.`]**

6.3 Phase 1: interface sweep

Figure 1 (the paper’s main figure): $S(\tau)$ curves per cell with intervals, one panel per axis. Table: c_{50} and c_{90} per cell, IQM over seeds. Regression of c_{50} on the effective per-step resolution $\min(\Delta_{\max}, \dot{q}_{\max} J/f) \cdot g(\alpha)$. **[TODO: Run and fill; 30 training runs. Blocked on the gate decision above.]**

6.4 Phase 2: controls

One table: c_{50} at θ_0 for baseline, reward arm, budget arm, Fourier arm; and c_{50} at $\Delta_{\max}=5$ mm with default vs. tuned hyperparameters. **[TODO: Run and fill; 17 training runs.]**

6.5 Failure attribution: how much of the residual failure is the scene?

At $\tau = c_{50}$, by construction, half the episodes fail. Two questions follow: can the failures be predicted from the sampled randomisation vector, and would the same physical scene fail again?

Prediction. A five-fold cross-validated classifier of failure on the thirteen sampled randomisation and scene variables reaches AUC 0.56–0.65 across the Phase 0 cells at $\tau = c_{50}$ and 0.53–0.65 at 45 mm — better than chance, but only just. Permutation importance puts the target’s azimuth and radius first in almost every cell, with the object’s sampled width next; the object’s mass, its friction, the table friction and the actuator gain scales sit at or below zero. Where the object has to be *put* in the workspace carries the signal; what it is made of does not.

Replay. A nested evaluation of π_{v2} — 200 randomisation draws, each replayed under 10 independent noise streams, 2000 episodes — separates the two sources of variance directly. The environment supports the split (`options={"dr_seed": k, "noise_seed": m}`), and the classifier for this cell is cross-validated with folds grouped by scene, without which replays of one scene appear on both sides of the split and the AUC is inflated by about 0.08. The one-way random-effects intra-class correlation of the binary outcome is $ICC = 0.175$ at $\tau = c_{50}$ and 0.113 at 45 mm.

Verdict against the pre-specified rule. The rule was that if $AUC < 0.6$ and $ICC < 0.1$ in every cell, the residual failures are noise and this section collapses to a paragraph. Neither threshold is quite met: about a sixth of the outcome variance at the floor is attributable to the scene, and a classifier can see a little of it. The honest statement is therefore the intermediate one — *at the floor, roughly 82% of whether a given episode succeeds is decided by observation and actuation noise rather than by the scene it was given*, and the minority that the scene decides is geometric rather than

dynamical. For H1 this is supporting context rather than evidence: a floor that is five parts noise to one part scene is not a floor that better coverage of the randomisation space would remove.

7 Discussion

[TODO: Write in full after Phases 1–2. The following is what the completed Phase 0 already licenses.]

Where the floor is located in the episode. The decomposition in Table 3 narrows the question considerably. The terminal error is essentially the tool-centre-point error at the last held step; post-release physics contributes real displacement (a 7.6 mm median) but almost no explanatory variance ($r^2 \approx 0.004$). Any account of the floor that appeals to settling, rolling or contact release is therefore the wrong account for this task. That is a necessary condition for H1, not a sufficient one: it localises the floor to the terminal approach, which is where $\Delta_{\max}$, α , f and $\dot{q}_{\max}$ act — but also where the observation noise, the IK residual and the policy’s own value estimate act.

How much room the interface hypothesis has. The kill gate bounds the simulator’s contribution at 1.9 mm of c_{50} across settings that a practitioner would consider reasonable. For H1 to be interesting, the interface sweep must move c_{50} by substantially more than that. It is worth saying now that the design has no guaranteed win: $\Delta_{\max}$ could move the floor by 2 mm and the paper would then be a null result about interfaces with an awkward simulator-sensitivity caveat attached. The honest framing in that case is a short negative note, and it is the framing this draft is prepared to adopt.

Relation to Xu et al. (2026). Their c for peg insertion and cuboid stacking is 2–3 mm; ours is 16 mm on a task with a post-hoc tolerance, a 30 mm action step and a low-cost 6-DoF arm. The numbers are not comparable and we do not compare them. What is comparable is the shape of the claim — that c is a property of the agent system rather than the task — and the mechanism we propose is a concrete instance of it that they could not observe, because they held the action interface fixed throughout.

Relation to Bronars et al. (2026) and Gyenes et al. (2026). Both remain open until Phase 2. If hyperparameter re-tuning recovers precision at small $\Delta_{\max}$, the interface sets the floor only for a fixed optimiser configuration, which is weaker but still the practically relevant statement for anyone who is not going to re-tune. If Fourier features move the floor at fixed θ , the representation explanation extends to RL with state observations and H1 must be restated jointly.

Practical reading. *What a practitioner should change first when a policy plateaus above the required tolerance — the interface, the reward, or the budget — is the paper’s intended takeaway and cannot be written before Phase 1.* What Phase 0 alone supports is narrower and still useful: before blaming the simulator, check where in the episode the error is created, and do not diagnose a floor from a 200-episode evaluation.

8 Limitations

All experiments are in simulation. The step-size axis is measured on two arms, two controllers and two tasks — an AgileX Piper under differential IK with a joint-velocity safety layer, and a Franka Panda under operational-space control with none — which bounds but does not remove the objection that the result is a property of one robot; the remaining axes and every control arm are measured on one arm, one task and one algorithm. The second environment’s task is a reach-and-grasp rather than a placement, for the reason given in Sec. 6.2, so the two floors are not the same physical quantity and are compared only in how they move with $\Delta_{\max}$. The solver arm bounds the dependence of c on the contact parameters we sweep, not on the contact model as such; it also found a non-zero dependence, which we report rather than round to nothing. A real-arm measurement remains the only complete answer, and we do not have one: Luo et al. (2025) obtain their precision on hardware, and whether the interface accounts for it there is a question only a lab with that hardware can settle. What this paper offers is the controlled version — two simulators, with the human, the reward classifier and the demonstration data removed, so the interface can be varied on its own. **[TODO: If the two-servo encoder rig is built, report it here as a partial hardware check on the step-size axis only, with the fixture-compliance calibration; otherwise state plainly that no hardware validation was performed.]** The observation is a ground-truth state; policies with visual observations may have an additional, representation-dominated floor. The interface family is Cartesian delta position through differential IK; joint-space and velocity interfaces

(Aljalbout et al., 2024; Azimi et al., 2026) may behave differently. The task’s tolerance is a post-hoc ruler rather than a physical constraint; *a stacking or insertion variant with a physical tolerance — a pedestal of radius r_{ped} slightly larger than the object, so that $\tau = r_{\text{ped}} - r_{\text{obj}}$ is a property of the scene — would remove this objection and is the natural second task.* Finally, the released MJCF has no joint damping, so the damping randomisation documented in earlier versions of this work was inert; the results here are for a model with zero joint damping, which is itself a modelling choice we did not make deliberately.

Reproducibility statement

The environment, MJCF, training and evaluation scripts, all checkpoints and every per-episode evaluation log are released at https://github.com/abdulahad2008/piper_simulation. **[TODO: Add a persistent DOI (Zenodo) and an anonymised mirror before submission.]** Every number in Sec. 6.1 is regenerated from the committed CSVs by

```
python reproduce.py -results results/precision -out paper/generated
```

which reads no simulator and needs no GPU. Package versions are pinned in `requirements.txt` and additionally recorded, together with the git hash, in the `*.meta.json` beside every results CSV and in every run’s `config.json`. `tests/test_interface_flags.py`, `tests/test_tolerance_semantics.py` and `tests/test_gymhil_interface.py` assert that every swept flag reaches both simulators, that the tolerance means what this paper says it means, and that `gym-hil`’s released interface constants are the ones Table 5 reports (75 tests in total). `CHANGELOG.md` documents every claim that changed between the August 2026 status documents and the archived numbers, including two that were simply wrong.

References

- Rishabh Agarwal, Max Schwarzer, Pablo Samuel Castro, Aaron Courville, and Marc G. Bellemare. Deep reinforcement learning at the edge of the statistical precipice. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2021.
- Takuya Akiba, Shotaro Sano, Toshihiko Yanase, Takeru Ohta, and Masanori Koyama. Optuna: A next-generation hyperparameter optimization framework. In *ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2019.
- Elie Aljalbout, Felix Frank, Maximilian Karl, and Patrick van der Smagt. On the role of the action space in robot manipulation learning and sim-to-real transfer. *IEEE Robotics and Automation Letters*, 2024. URL <https://arxiv.org/abs/2312.03673>. arXiv:2312.03673.
- Elie Aljalbout, Jiaxu Xing, Angel Romero, Iretiayo Akinola, Caelan Garrett, Eric Heiden, Abhishek Gupta, Tucker Hermans, Yashraj Narang, Dieter Fox, Davide Scaramuzza, and Fabio Ramos. The reality gap in robotics: Challenges, solutions, and best practices. *Annual Review of Control, Robotics, and Autonomous Systems*, 9, 2026. URL <https://arxiv.org/abs/2510.20808>. arXiv:2510.20808.
- Michel Aractingi, Adil Zouitine, and Hugging Face. `gym-hil`: Human-in-the-loop reinforcement learning suite. <https://github.com/huggingface/gym-hil>, 2025. Apache-2.0; MuJoCo Franka Panda environments used as this paper’s second environment.
- Seyed Alireza Azimi, Homayoon Farrahi, Abhishek Naik, Colin Bellinger, and A. Rupam Mahmood. Benchmarking action spaces in reinforcement learning for vision-based robotic manipulation. *arXiv preprint arXiv:2606.18594*, 2026. URL <https://arxiv.org/abs/2606.18594>.
- Antonia Bronars, Younghyo Park, and Pulkit Agrawal. Tune to learn: How controller gains shape robot policy learning. *arXiv preprint arXiv:2604.02523*, 2026. URL <https://arxiv.org/abs/2604.02523>.
- Hunter L. Brown, Geoffrey Hollinger, and Stefan Lee. Learning hybrid-control policies for high-precision in-contact manipulation under uncertainty. *arXiv preprint arXiv:2604.19677*, 2026. URL <https://arxiv.org/abs/2604.19677>.

-
- Qian Chen, Junqiao Zhao, et al. ACSAC: Adaptive chunk size actor-critic with causal transformer q-network. *arXiv preprint arXiv:2605.11009*, 2026. URL <https://arxiv.org/abs/2605.11009>.
- Yuchun Feng, Jinliang Zheng, Zhihao Wang, Dongxiu Liu, Jianxiong Li, Jiangmiao Pang, Tai Wang, and Xianyuan Zhan. Demystifying action space design for robotic manipulation policies. *International Conference on Machine Learning (ICML)*, 2026. URL <https://arxiv.org/abs/2602.23408>. arXiv:2602.23408.
- Balázs Gyenes, Emiliyan Gospodinov, Jan Frieling, Enrico Krohmer, Nicolas Schreiber, Xiaogang Jia, Niklas Freymuth, and Gerhard Neumann. Fourier features let agents learn high precision policies with imitation learning. *International Conference on Machine Learning (ICML)*, 2026. URL <https://arxiv.org/abs/2606.12334>. arXiv:2606.12334.
- Tuomas Haarnoja, Aurick Zhou, Pieter Abbeel, and Sergey Levine. Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. In *International Conference on Machine Learning (ICML)*, 2018.
- Xiaoyi Hu, Qiao Sun, Bailin He, Haojie Liu, Xueyi Zhang, Chunpeng Lu, and Jiangwei Zhong. Impact of static friction on sim2real in robotic reinforcement learning. *arXiv preprint arXiv:2503.01255*, 2025. URL <https://arxiv.org/abs/2503.01255>.
- Tianchong Jiang, Xiangshan Tan, Samuel Wheeler, Luzhe Sun, Tewodros W. Ayalew, and Matthew Walter. What are we actually benchmarking in robot manipulation? *arXiv preprint arXiv:2606.04233*, 2026. URL <https://arxiv.org/abs/2606.04233>.
- Oleg Kaidanov, Firas Al-Hafez, Yusuf Süvari, Boris Belousov, and Jan Peters. The role of domain randomization in training diffusion policies for whole-body humanoid control. In *CoRL 2024 Workshop on Whole-Body Control and Bimanual Manipulation*, 2024. URL <https://arxiv.org/abs/2411.01349>. arXiv:2411.01349.
- Michael Kaup, Cornelius Wolff, Hyerim Hwang, Julius Mayer, and Elia Bruni. A review of nine physics engines for reinforcement learning research. *arXiv preprint arXiv:2407.08590*, 2024. URL <https://arxiv.org/abs/2407.08590>.
- Hadas Kress-Gazit, Kunimatsu Hashimoto, Naveen Kuppaswamy, Paarth Shah, Phoebe Horgan, Gordon Richardson, Siyuan Feng, and Benjamin Burchfiel. Robot learning as an empirical science: Best practices for policy evaluation. *arXiv preprint arXiv:2409.09491*, 2024. URL <https://arxiv.org/abs/2409.09491>.
- Qiyang Li, Zhiyuan Zhou, and Sergey Levine. Reinforcement learning with action chunking. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025. URL <https://arxiv.org/abs/2507.07969>. arXiv:2507.07969.
- Jianlan Luo, Charles Xu, Jeffrey Wu, and Sergey Levine. Precise and dexterous robotic manipulation via human-in-the-loop reinforcement learning. *Science Robotics*, 2025. doi: 10.1126/scirobotics.ads5033. URL <https://doi.org/10.1126/scirobotics.ads5033>. arXiv:2410.21845.
- Fabio Muratore, Fabio Ramos, Greg Turk, Wenhao Yu, Michael Gienger, and Jan Peters. Robot learning from randomized simulations: A review. *Frontiers in Robotics and AI*, 9, 2022. URL <https://www.frontiersin.org/articles/10.3389/frobt.2022.799893/full>.
- Andrew Patterson, Samuel Neumann, Martha White, and Adam White. Empirical design in reinforcement learning. *Journal of Machine Learning Research*, 25(318), 2024. URL <https://jmlr.org/papers/v25/23-0183.html>.
- Antonin Raffin, Ashley Hill, Adam Gleave, Anssi Kanervisto, Maximilian Ernestus, and Noah Dormann. Stable-baselines3: Reliable reinforcement learning implementations. *Journal of Machine Learning Research*, 22(268), 2021.
- David Snyder, Asher James Hancock, Apurva Badithela, Emma Dixon, Patrick Miller, Rares Andrei Ambrus, Anirudha Majumdar, Masha Itkina, and Haruki Nishimura. Is your imitation learning policy better than mine? policy comparison with near-optimal stopping. In *Robotics: Science and Systems (RSS)*, 2025. URL <https://arxiv.org/abs/2503.10966>. arXiv:2503.10966.

Matthew Tancik, Pratul P. Srinivasan, Ben Mildenhall, Sara Fridovich-Keil, Nithin Raghavan, Utkarsh Singhal, Ravi Ramamoorthi, Jonathan T. Barron, and Ren Ng. Fourier features let networks learn high frequency functions in low dimensional domains. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

Emanuel Todorov, Tom Erez, and Yuval Tassa. MuJoCo: A physics engine for model-based control. In *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2012.

Edwin B. Wilson. Probable inference, the law of succession, and statistical inference. *Journal of the American Statistical Association*, 22(158):209–212, 1927.

Cuijie Xu, Yuanfan Xu, Min Xue, Jianjie Lin, Jian Wang, Xudong Zhang, Yu Wang, and Jincheng Yu. The curse of precision: A data scaling law for high-precision robotic manipulation. In *IEEE International Conference on Robotics and Automation (ICRA)*, 2026. URL <https://arxiv.org/abs/2607.23108>. arXiv:2607.23108.

A Reward specification

[TODO: Table of every reward term, weight and trigger condition, generated from `piper_rl/config.py` at the commit used.]

B Domain randomisation specification

[TODO: Table of every randomised parameter and range, generated from `config.py`, including the joint-damping no-op correction and the `joint_damping_is_noop` flag now emitted by the randomiser.]

C Per-cell training curves

[TODO: Learning curves for every cell and seed.]